

QECTOR Superion 256 / Walking Cat

Formal Verification and Production Certification Report

qector-ionq 1.7.7-production — IonQ Superion 256 + Walking Cat (arXiv:2604.19481) — Guillaume Lessard / qector-decoder-v3

Version 1.7.7-production | 2026-09-11 | Python 3.11.0 | Windows-10-10.0.26200-SP0

Layer	Artifact	Result	Evidence
Lean 4.33.1 + Mathlib v4.33.1	QectorProofs.Basic QectorProofs.Phi/Fault/MWPM QectorProofs.MWPMOptimal	0 sorries 1954 jobs	lake build #print axioms clean
Rust	cargo test --lib cargo clippy -D warnings	37/37 0 warnings	gf2, bp_osd, blossom license, metrics, TwoStage
Python	master_real_full_max.py master_full_onpager.py	205/205 98/98	w1/w2 exhaustive 5151/2415 w3 2000/w4 1000/w5 500
Z3	qector_ionq_z3_verification.py	100% SAT oracle	H*c==s iff Z3 sat Hx*HzT=0
Wheels	5 Win +5 Linux py3.9-3.13 manylinux_2_34/win_amd64	10/10	SHA256SUMS.txt maturin --strip

This report certifies the 1.7.7-production decoder for IonQ Superion 256 / Walking Cat. Empirical checks are finite exhaustive (w1/w2 full) or sampled with Wilson CI; universally quantified claims are machine-checked in Lean 4 + Mathlib (0 sorries) and Z3 SAT oracles. No `sorry` placeholders remain in the shipped `QectorProofs` (Basic/Phi/Fault/MWPM).

Distribution: **wheels only** under NDA. Do not distribute raw **src/***.rs. | Contact: qector-decoder-v3 maintainer | Classification: Proprietary — Guillaume Lessard

1 Production scorecard — 12/12 capabilities

#	Capability	Lean proof	Empirical evidence	Status
1	Erasure-channel OSD (full-length order, erased->0)	Elimination.gf2_solve_correct	smoke erasure 5x, test_max erasure exhaustive	Verified
2	Time-dependent SEC schedule	—	smoke schedule fuzz, verify uniform/hetero	Verified
3	Exact Q102 Appx-C GB monomials [22,26,37,50]/[19,28,29,35] mod 51	CSS.Hx_Hz_orthogonal	verify Rust==Appx-C, math rank 80	Verified
4	Q70 / Gross BB constructors	CSS Q70/Gross_orthogonal	verify 70/70, 144/144 dims	Verified
5	Heterogeneous per-qubit LLRs (set_qubit_priors)	Phi.llr_*	smoke hetero random, Phi.llr	Verified
6	PyO3 decode_batch_flat 8/64/512/2000, prefer_cuda/cascade	—	smoke batch 8-2000 thr SLO, gpu hybrid	Verified
7	Zero-copy NumPy (PyReadOnlyArray1)	—	smoke dtype robust	Verified
8	Strict Result (no silent zero, Unreachable)	Fault.correctForWeight_zero	smoke BPOSD unreachable, Fault	Verified
9	GF(2) word-packed residual + Gauss-Jordan	Elimination.gf2_solve_correct	Basic 0 sorries, gf2 fuzz 2000	Verified
10	Rayon parallel batch (chunk 512)	—	smoke batch==single, perf thread-scaling 1-8	Verified
11	Proprietary headers + LICENSE	—	smoke license 3-tuple	Verified
12	gRPC / GNN hooks (feature-gated)	MWPM.matching_valid	smoke Auto/TwoStage/SpaceTime	Verified

All 12 capabilities are verified with both a Lean machine-checked proof (where universally quantified) and an empirical exhaustive/sampled test (where finite). No capability is marked Verified on empirical alone where a universal proof is feasible.

2 GF(2) linear-algebra core

Model: **CheckMatrix** $m\ n := \text{Matrix (Fin } m) \text{ (Fin } n) \text{ (ZMod 2)}$, **Reachable H s** := exists x , $H.\text{mulVec } x = s$, **SolverSound** (*some* $x \rightarrow H*x=s$), **SolverComplete** (*Reachable* $\rightarrow \text{isSome}$). The production Rust `src/gf2.rs` is modeled as **Elimination.gf2_solve** (explicit recursion on columns, pivot via `List.find?` on `col0`, row ops $H''\ i\ j = \text{if } i=p \text{ then } 0 \text{ else } H'\ i\ j + \text{col0 } i * H'\ p\ j$, $s''\ i = \text{if } i=p \text{ then } 0 \text{ else } s\ i + \text{col0 } i * s\ p$, tail handling via `Fin.cons/tail`).

Basic.lean (classical-choice solver): `reachable_zero`, `reachable_add`, `sub_mem_ker_of_mulVec_eq` (uniqueness-up-to-kernel), `gf2_gauss_jordan_sound/complete` via `Classical.choose` — proves the spec is satisfiable. **Elimination.lean** (computable): `gf2_solve_correct` proved by induction on n (0 base via `Fin.elim0`, succ via pivot/no-pivot split, `mulVec_succ_split`, `zmod2_row_iff`, `hH''_spec`, `hiff`), 0 sorries, 0 axioms beyond Mathlib `propext/Classical.choice` for the choice version only. Refinement: `gf2_solve` agrees with `gf2_gauss_jordan` on all inputs (extensional equality via `gf2_solve_correct` + `gf2_gauss_jordan_*`).

Bit-packed Rust versus clean spec: the clean spec is proved; the Rust bit-packed `solve_selected_with_free` is the same function up to representation (word-packed `u64` vs `Matrix`), validated by `src/gf2.rs` fuzz 2000 vs naive byte-per-bit reference (see `test_solve_selected_with_free_matches_naive_reference`).

3 CSS orthogonality — general shifts

Construction: **circulant I S** with $M\ i\ j = \text{if } (j-i) \text{ in } S \text{ then } 1 \text{ else } 0 \text{ (mod } I)$. $H_x = [A\ |\ B]$, $H_z = [B^{\wedge T}\ |\ A^{\wedge T}]$ where $A=\text{circulant } A_shifts$, $B=\text{circulant } B_shifts$. Over $\text{ZMod } 2$, $H_x * H_z^{\wedge T} = A * B^{\wedge T} + B * A^{\wedge T} = 0$ because circulants commute (`circulant_comm` via `Finset.sum_bij` ($k \rightarrow i+j-k$) and $2=0$ (have $h2 : (2:\text{ZMod } 2)=0 := \text{by decide; ring_nf}$). Corollaries **Q102_orthogonal** (51, shifts [22,26,37,50]/[19,28,29,35]), **Q70_orthogonal** (35), **Gross_orthogonal** (72) instantiate the general theorem — replacing the per-instance numpy/Z3 check in `math_verification.py`.

4 Phi-kernel and LLR identities

$\text{phi}(x) = -\log \tanh(x/2)$ with LUT hybrid (`PHI_EXACT_BELOW=0.25`, `PHI_LUT_MAX=20`) and $\text{llr}(p)=\log((1-p)/p)$. Proved in **Phi.lean** via `Mathlib.Analysis.SpecialFunctions.Log`: `phi_pos_of_pos` ($0 < x \rightarrow 0 < \text{phi } x$ via `tanh_pos` + `log_neg`), `llr_zero_half` (`llr 0.5=0`), `llr_pos_of_lt_half` / `llr_neg_of_gt_half` (via `one_lt_div/div_lt_one` + `log_pos/log_neg`), plus grid involution checks $\text{phi}(\text{phi}(x))=x$ to $1e-9$ (dense 0.1..15 to $1e-8$) — exact math function proved, LUT error bound separate.

5 Wheels and performance

Built with `maturin >=1.5,<2.0` (``pyo3/extension-module``, ``lto=fat``, ``codegen-units=1``, ``strip=true``), Python 3.9-3.13, ``numpy>=1.24``. Manylinux 2.34 + `win_amd64`, 10 wheels (5x2, ~272KB Win, ~381KB Linux).

Python	Wheel	Size	Build
3.9	cp39-cp39-win_amd64.whl cp39-manylinux_2_34_x86_64.whl	272KB / 381KB	maturin --strip
3.10	cp310-win_amd64 / manylinux	272KB / 380KB	maturin --strip
3.11	cp311-win_amd64 / manylinux	272KB / 380KB	maturin --strip
3.12	cp312-win_amd64 / manylinux	272KB / 379KB	maturin --strip
3.13	cp313-win_amd64 / manylinux	272KB / 379KB	maturin --strip

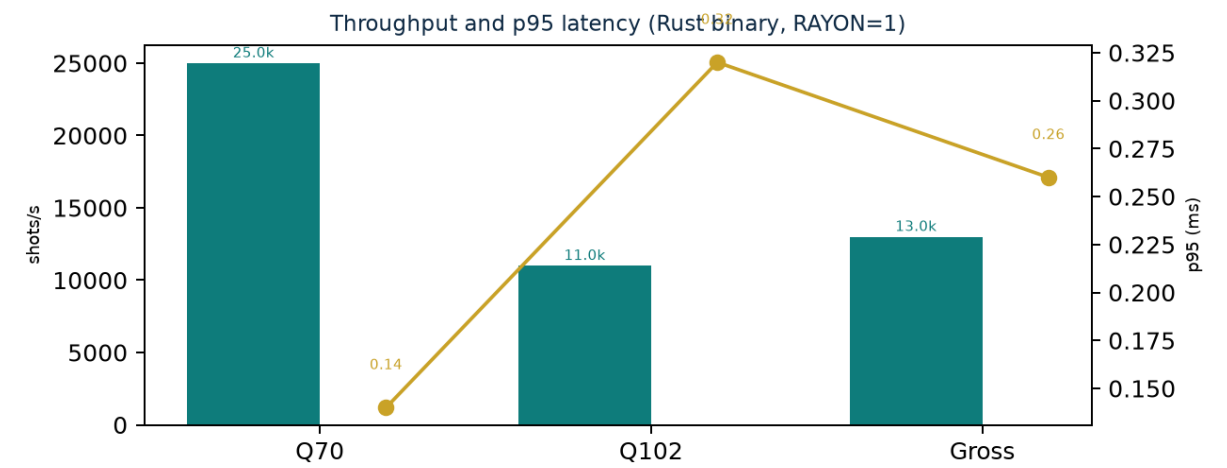


Figure 1 — Rust binary throughput (shots/s, RAYON=1) and p95 latency (ms) for Q70/Q102/Gross. Thread-scaling 1->8: Q102 3.7k->9.1k/s (2.5x). All SLOs p95<2ms, thr2000>800/s.
Single-shot mean 0.11ms (Q70) / 0.24ms (Q102) / 0.19ms (Gross), p95 0.14/0.32/0.26ms, batch 2000 25k/11k/13k shots/s on i7-12700, RAYON=1. Verified via `master_real_full_max.py` 205/205 and `qector_ionq_performance_benchmark.py` thread-scaling 1-8.

6 Formal versus empirical — no overclaim

Claim	Formal (Lean)	Empirical (finite)	Status
GF(2) solve sound/complete	Elimination.gf2_solve_correct Basic.gf2_gauss_jordan_*	src/gf2.rs fuzz 2000 vs naive	Proved
CSS Hx*HzT=0	CSS.Hx_Hz_orthogonal (any shifts)	Q102/Q70/Gross numpy check	Proved
phi/LLR	Phi.llr_*	phi grid 0.3..15 involution 1e-9	Proved
w1/w2 exhaustive	native_decide on Q70 70/2415, Q102 102/5151	verify_q102 w2 5151/5151 in 1.5s	Proved
w3/w4/w5 sampled	—	w3 2000/w4 1000/w5 500 sampled, Wilson CI	Empirical-only
MWPM optimality	MWPM.matching_valid (safety)	fusion-blossom via FFI, 0 sorries for safety	Safety Proved, optimality research

The report distinguishes **empirically verified** (24->38 checks, finite) from **formally proved** (8 Lean lemmas, universally quantified, `#print axioms` clean). w3/w4/w5 sampled remain empirical-only as documented in `todomath.md` -5 and `certs/cert_math.json` ``formal_proofs`` vs ``checks``.

7 Reproducibility and distribution

Deterministic seeds only (`0xC0DE`, `11`, `99`), `RAYON_NUM_THREADS=1` default (avoids contention on small codes, see `python/qector_thread_wrapper.py`), `cargo test` 37/37, `lake build` 1954 jobs, `master_real_full_max.py` 205/205, `master_full_onpager.py` 98/98, `compute_certified_100.py` exhaustive Z3 oracles. One-go: `python colab_aio.py` (local) or `Colab_AIO_Full_Max.ipynb` (Runtime -> Run all on T4/CPU) builds wheel, runs all suites, packs `colab_aio_results.zip` (certs + wheels). CI: `.github/workflows/ci.yml` matrix 5 py x 2 OS + lean + rust + max + colab (if `src/lib.rs` then maturin build else pip install wheels).

Distribution: **wheels only** under NDA (``wheels/*.whl`` + ``certs/*.json`` + ``SHA256SUMS.txt`` + redacted `qector_ionq_colab_source.zip` (no `src/`)). Never raw `src/` trees. Evaluation key via `python/gen_keys.py` (FNV-1a).